PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 02 -

APROFUNDANDO NO TERRAFORM: VARIÁVEIS, OUTPUTS E STATE

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 13
O QUE VOCÊ VIU NESTA AULA? 14
REFERÊNCIAS 15

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VEM POR AÍ?

Na primeira aula, vimos que o Terraform e o OpenTofu permitem transformar infraestrutura em código. Até agora, trabalhamos de forma mais direta, sem explorar recursos que realmente dão poder e escalabilidade a esses projetos. Nesta aula, você vai aprender a usar variáveis para parametrizar configurações, outputs para compartilhar informações entre módulos e o temido — mas essencial — state, o arquivo que registra a verdade sobre a sua infraestrutura.

Este é o ponto em que o Terraform deixa de ser apenas uma ferramenta curiosa e se torna uma peça essencial do seu arsenal de DevOps. Cada conceito aqui vai ajudar você a escrever um código mais limpo, reaproveitável e seguro. Vamos juntos(as) dar esse próximo passo.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

HANDS ON

Chegou a hora de traduzir teoria em prática. Nesta etapa, você vai acompanhar em vídeo a construção de um projeto Terraform que utiliza variáveis, outputs e o state para provisionar recursos na AWS. A ideia é que você veja como esses conceitos se aplicam no dia a dia e como tornam o código mais flexível e reaproveitável. Não se preocupe em decorar cada comando: o objetivo do Hands On é mostrar o fluxo completo de criação, ajuste e reaplicação de infraestrutura.

Nos vídeos, vamos começar definindo variáveis para parametrizar valores importantes, como tipo de instância e região da AWS. Em seguida, veremos como os outputs permitem extrair informações úteis automaticamente, como o IP público de uma máquina provisionada. Por fim, você verá na prática como o arquivo de state funciona: aplicaremos mudanças, consultaremos os recursos e configuraremos o armazenamento remoto para garantir segurança e colaboração em equipe.

Essa parte prática é essencial para que os conceitos façam sentido de verdade. Assista aos vídeos com calma, pause quando precisar e repita os passos no seu próprio ambiente. É comum errar no início, mas é justamente nesses erros que o aprendizado se fortalece. Aproveite o Hands On como um laboratório: explore, teste e compare os resultados com o que aparece na sua tela.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SAIBA MAIS

Uma das grandes forças do Terraform (e do OpenTofu) está na sua capacidade de abstrair detalhes de configuração e padronizar projetos. Se na primeira aula vimos como criar recursos de forma declarativa, agora o foco é aprender a parametrizar, expor informações úteis e controlar o estado da infraestrutura. Esses três elementos — variáveis, outputs e state — são o que diferenciam projetos de laboratório de projetos que podem ser usados em ambientes reais.

Imagine que você precisa criar dez instâncias em diferentes regiões da AWS. Se você codificar tudo diretamente em arquivos `.tf`, deverá repetir várias vezes os mesmos valores, mudando apenas pequenas partes. Isso torna o projeto difícil de manter.

O Terraform segue o princípio “Don't Repeat Yourself (DRY)”, e as variáveis são a principal forma de evitar repetição. Em vez de espalhar valores fixos pelo código, declaramos variáveis centralizadas, trazendo flexibilidade, reaproveitamento e organização.

Tipos de variáveis no Terraform

As variáveis no Terraform podem ser tipadas, o que garante maior previsibilidade e segurança no código. Definir tipos evita erros comuns, como passar um número onde se esperava uma string.

Os principais tipos aceitos são:

1. String: representa valores de texto, como nomes, regiões ou IDs.

```
variable "aws_region" {
  type        = string
  default     = "us-east-1"
  description = "Região da AWS"
}
}
```

2. Number: representa números inteiros ou decimais.

```
variable "instance_count" {
  type        = number
  default     = 2
  description = "Número de instâncias EC2 a serem criadas"
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
}
```

3. Bool (booleano): aceita `true` ou `false`.

```
variable "instance_count" {
variable "enable_monitoring" {
  type        = bool
  default     = true
  description = "Habilitar monitoramento detalhado da instância”
}
```

4. List (listas de valores ordenados): são conjuntos ordenados, ideais para armazenar múltiplos valores.

```
variable "availability_zones" {
  type        = list(string)
  default     = ["us-east-1a", “us-east-1b"]
  description = "Lista de zonas de disponibilidade”
}
```

Uso no código:

```
subnet_id = var.availability_zones[0]
```

5. Map (pares chave-valor): permite armazenar dados em formato de dicionário.

```
variable "instance_types" {
  type = map(string)
  default = {
    dev  = “t2.micro"
    prod = “t3.large"
  }
  description = "Tipos de instância por ambiente”
}
```

Uso no código:

```
instance_type = var.instance_types["prod"]
```

6. Object (estruturas complexas): permite agrupar diferentes atributos em uma única variável.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
variable "app_config" {
  type = object({
    name     = string
    replicas = number
    enabled  = bool
  })
  default = {
    name     = “minha-app"
    replicas = 3
    enabled  = true
  }
}
```

7. Set (conjunto não ordenado de valores únicos): semelhante à lista, mas não garante ordem e não aceita duplicatas.

```
variable "unique_tags" {
  type        = set(string)
  default     = ["web", "api", “db"]
  description = "Conjunto de tags únicas”
}
```

Exemplo com vários tipos:

```
variable "replicas" {
  type    = number
  default = 3
}

variable "enable_https" {
  type    = bool
  default = true
}

variable "zones" {
  type    = list(string)
  default = ["us-east-1a", "us-east-1b"]
}

variable "app_config" {
  type = object({
    name     = string
    version  = string
    replicas = number
  })
  default = {
    name     = "minha-app"
    version  = "1.0.0"
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
    replicas = 2
  }
}
```

Por que isso importa?

O uso correto de tipos:

- Evita erros durante o terraform apply.
- Facilita validação de valores (exemplo: não aceitar string em campo numérico).
- Aumenta a clareza do código para toda a equipe.
- Permite criar variáveis complexas que substituem configurações extensas, mantendo o código mais limpo.

Além disso, é possível definir variáveis sem tipo (chamadas de *dynamic typing*). Nesse caso, o Terraform infere o tipo com base no valor. Porém, em projetos profissionais, sempre é recomendado declarar explicitamente o tipo para evitar comportamentos inesperados.

Arquivos .tfvars

Os arquivos `.tfvars` servem para definir valores de variáveis em diferentes ambientes. Isso é útil para separar configurações de dev, staging e prod, sem alterar o código principal.

Exemplo de `dev.tfvars`:

```
aws_region    = "us-east-1"
instance_type = "t2.micro"
```

Exemplo de `prod.tfvars`:

```
aws_region    = "us-east-1"
instance_type = "t3.large"
```

Na execução, basta informar o arquivo:

```
terraform apply -var-file="prod.tfvars"
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Isso garante que a mesma infraestrutura possa ser replicada em vários ambientes, mudando apenas os valores.

Boas práticas de organização

- Separar lógica de valores: mantenha valores dinâmicos em `.tfvars`, e apenas defaults ou documentação em `variables.tf`.
- Descrever variáveis: sempre use `description` para explicar a finalidade.
- Escolha nomes claros, como `aws_region` em vez de apenas `region`.
- Versionar tfvars por ambiente: `dev.tfvars`, `prod.tfvars` etc.
- Não versionar secrets: use soluções seguras como Vault, SSM Parameter Store ou GitHub Secrets.

Outputs: comunicação entre módulos e projetos

Se as variáveis representam entradas, os outputs representam saídas. Eles são fundamentais para compartilhar informações entre módulos.

Os outputs são como uma "resposta final" do Terraform depois de aplicar a configuração. Eles servem para expor informações úteis de recursos criados, como IPs, IDs e URLs. Isso é essencial quando trabalhamos com módulos: um módulo pode gerar um valor que será usado por outro.

Veja como expor o IP público e o ID da instância criada:

Exemplo de `outputs.tf`:

```
output "instance_public_ip" {
  description = "IP público da instância EC2”
  value       = aws_instance.web.public_ip
}

output "instance_id" {
  description = "ID da instância EC2 criada"
  value       = aws_instance.web.id
}
```

Quando rodamos `terraform apply`, o Terraform exibirá automaticamente esses valores no final da execução. Em projetos maiores, esses outputs podem ser consumidos por outros módulos, criando encadeamentos de infraestrutura.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O resultado será algo como:

```
Outputs:

instance_public_ip = "3.94.102.45"
instance_id = "i-0a6002c2eb97285b7"
```

Encadeamento de recursos

Módulos podem consumir outputs de outros módulos. Exemplo: um módulo de rede expõe o ID da VPC, e outro módulo de EC2 usa esse ID.

```
# outputs do módulo de rede
output "vpc_id" {
  value = aws_vpc.main.id
}
```

No módulo de instâncias:

```
module "network" {
  source = "./network"
}

resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = var.instance_type
  vpc_id        = module.network.vpc_id
}
```

Isso cria um encadeamento natural entre módulos.

Exemplos práticos:

1. Expor a URL de um Load Balancer e consumi-la em outro módulo.
2. Exportar credenciais de um banco e passá-las para uma aplicação.
3. Encadear IDs de subnets para configurar Auto Scaling Groups.

Outputs, quando bem usados, transformam módulos isolados em blocos reutilizáveis.

State: o coração do Terraform

O state é um dos pontos mais críticos do Terraform. Sempre que você aplica uma configuração, o Terraform mantém um arquivo chamado terraform.tfstate, que

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

descreve o estado atual dos recursos. O state é o mecanismo que permite ao Terraform saber o que existe e o que precisa ser criado/atualizado/destruído.

Um exemplo: se você mudar o tipo de instância no código, o Terraform compara o estado atual (registrado no `tfstate`) com o estado desejado (no código) e decide aplicar a mudança. Sem esse arquivo, ele não teria como saber a diferença.

Em ambientes individuais, o state pode ser armazenado localmente. Mas em equipes, isso gera problemas: duas pessoas podem aplicar mudanças ao mesmo tempo e corromper o arquivo. Para evitar isso, usamos backends remotos, como um bucket S3 na AWS.

O arquivo `terraform.tfstate` é gerado após o `apply` e contém informações detalhadas da infraestrutura. Exemplo simplificado:

```
{
  "resources": [
    {
      "type": "aws_instance",
      "name": "web",
      "instances": [
        {
          "attributes": {
            "id": "i-0abcd1234efgh5678",
            "ami": "ami-08c40ec9ead489470",
            "instance_type": "t2.micro"
          }
        }
      ]
    }
  ]
}
```

Se o state for perdido ou corrompido, o Terraform perde a visão da infraestrutura, podendo recriar recursos indevidamente.

Local vs. remoto

- Local: o state é armazenado no diretório do projeto. Bom para estudos, mas perigoso em times.
- Remoto: armazenado em backends como S3, GCp, Azure Blob Storage. Ideal para colaboração e segurança.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Quando múltiplos usuários aplicam mudanças ao mesmo tempo, há risco de corrupção. Por isso usamos locking. Na AWS, o lock é feito com DynamoDB. Na GCP, o backend GCS já tem mecanismo de lock.

Essa prática garante que apenas um `apply` rode por vez, evitando conflitos. No arquivo `backend.tf`, podemos configurar o Terraform para usar o S3 e um DynamoDB para lock do state:

```
# backend.tf
terraform {
  backend "s3" {
    bucket         = "meu-bucket-terraform-state"    # Nome do bucket S3
    key                = "infra/aula2/terraform.tfstate" # Caminho do arquivo state
    region          = "us-east-1"
    dynamodb_table = "terraform-locks"             # Usado para lock
    encrypt        = true                          # Ativa criptografia
  }
}
```

Essa configuração garante que:

1. O state será armazenado em nuvem, acessível a todos da equipe.
2. O DynamoDB evitará que dois engenheiros rodem `terraform apply` ao mesmo tempo, impedindo conflitos.
3. O arquivo será criptografado para maior segurança.

Exemplo de back-end no Google Cloud:

```
terraform {
  backend "gcs" {
    bucket  = "meu-bucket-gcp"
    prefix  = "terraform/state"
  }
}
```

Variáveis tornam o código flexível e reutilizável, outputs permitem que módulos conversem entre si e o state garante que o Terraform saiba o que já existe antes de aplicar mudanças. Juntos, esses três elementos são a fundação para qualquer projeto de IaC em produção.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

MERCADO, CASES E TENDÊNCIAS

O uso de variáveis, outputs e state no Terraform pode parecer apenas detalhe técnico, mas na prática são justamente esses recursos que permitem que empresas escalem seus projetos de IaC. Cada vez mais organizações tratam infraestrutura como código versionado e compartilhado em times, e não como algo manual ou isolado. Esse movimento tem transformado a forma como o mercado de tecnologia constrói e mantém aplicações em produção.

Um exemplo relevante vem da Spotify, que em sua jornada para maior eficiência em escala global passou a adotar fortemente o conceito de infraestrutura declarativa. No case “Fleet Management at Spotify – The Path to Declarative Infrastructure”, a empresa mostra como o uso de Terraform, aliado a boas práticas de gestão de state remoto, permitiu reduzir riscos e aumentar a confiabilidade dos serviços. Antes, o uso de states locais causava inconsistências e até falhas em deploys. Com a centralização e automação do gerenciamento, acompanhada de bloqueios para evitar concorrência, a empresa conseguiu elevar o nível de colaboração entre engenheiros e tornar a infraestrutura mais previsível e segura.

Do ponto de vista de tendências, podemos observar o crescimento de ferramentas complementares como o Terragrunt, que adiciona camadas de organização e reutilização sobre o Terraform, ajudando a lidar com múltiplos ambientes de forma mais simples.

Se você quiser se aprofundar nesse tema, recomendo o livro “Terraform: Up and Running” de Yevgeniy Brikman (O’Reilly, 3ª edição, 2022), que traz exemplos práticos de como organizar variáveis, outputs e state em projetos reais. Outra ótima referência é o HashiConf conferência anual da HashiCorp disponível em vídeos no YouTube, onde empresas compartilham seus cases de uso de Terraform em escala.

“Fleet Management at Spotify (Part 2): The Path to Declarative Infrastructure. Spotify Engineering, 2023. Disponível aqui.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu a trabalhar com três pilares fundamentais do Terraform: variáveis, outputs e state. Entendeu como variáveis tornam o código mais flexível e organizado, permitindo a criação de ambientes diferentes sem duplicação. Viu como os outputs ajudam a compartilhar informações entre módulos e projetos, tornando a infraestrutura modular e reaproveitável. Por fim, compreendeu a importância do state como o “mapa” da sua infraestrutura, incluindo os riscos de armazená-lo localmente e as vantagens de usar backends remotos com mecanismos de bloqueio.

Agora você tem em mãos as bases necessárias para escrever projetos Terraform que vão além do básico, com capacidade de crescer em escala, de serem reutilizados e, principalmente, seguros para o trabalho em equipe.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

REFERÊNCIAS

HASHICORP. **Backends.** 2024. Disponível em: https://developer.hashicorp.com/terraform/language/settings/backends. Acesso em: 21 out. 2025.

HASHICORP. **Input Variables.** 2024. Disponível em: https://developer.hashicorp.com/terraform/language/values/variables. Acesso em: 21 out. 2025.

HASHICORP. **Output Values.** 2024. Disponível em: https://developer.hashicorp.com/terraform/language/values/outputs. Acesso em: 21 out. 2025.

HASHICORP. **State.** 2024. Disponível em: https://developer.hashicorp.com/terraform/language/state. Acesso em: 21 out. 2025.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

PALAVRAS-CHAVE

variaveis . terraform. infraestrutura.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com